Numerical Methods & Applied Programming (ENG2124)

Python Basics

Dr Ian (Yinglong) He

A: 14b AC 03

E: yh0015@surrey.ac.uk

T: 01483 688784

Overview

Structure

- This part (*Applied Programming*) will consist of 4 sections:
  - Python basics
  - NumPy for arrays
  - Pandas for datasets
  - Matplotlib for visualization

  } *Python packages for data science*

Assessment method

- 30% - Programming class test (multiple choice questions).

Course Materials → Assessment Information → Applied Programming Class Test

Overview

Teaching materials

- Materials have been made available online in advance (**SurreyLearn**), including:
  - Handouts (PPTs)
  - Tutorial sheets, including questions & answers (delayed release)
  - Python glossary
  - Mock programming test (same format as a real one)

Course Materials → Introduction to the Module → Your Learning: Applied Programming Part

Overview

Discussion forum

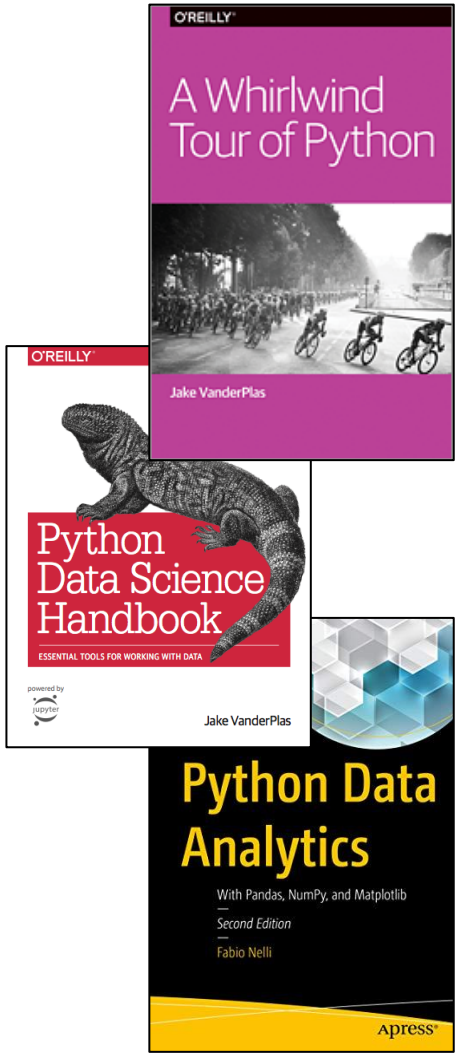
- There is a discussion forum set up in **SurreyLearn** for questions about the Python part. Please use this for any questions you have.
  - An area for general questions
  - An area for questions specifically about Python programming

Communication → Discussions → Applied Programming Forum

Overview

Recommended reading

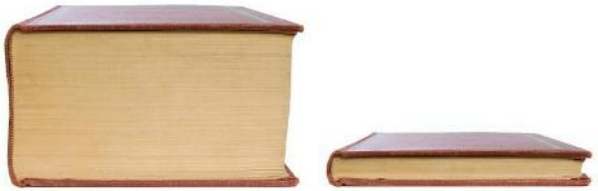
- Jake VanderPlas. *A Whirlwind Tour of Python*. 2016. (ISBN: 978-1491964644) https://jakevdp.github.io/WhirlwindTourOfPython/
- Jake VanderPlas. *Python Data Science Handbook: Essential Tools for Working with Data*. 2016. (ISBN: 978-1491912058) https://jakevdp.github.io/PythonDataScienceHandbook/
- Fabio Nelli. *Python Data Analytics*. 2nd Edition. 2018. (ISBN: 978-1484239124)



General advice

- Focus on the "common" aspects (e.g., data types, control flows, functions, key packages) of Python programming.
- Learning a language is all about practice.
- Google and ChatGPT are your best friends.

My code



My code without code from *Stack Overflow* and *GitHub*

Content

- What is Python?
- Python IDEs
- Basic syntax
- Data types
  - Simple: int, float, string, boolean
  - Collection: list, tuple, dictionary, set
- Operators: *arithmetic*, *assignment*, *comparison*, *boolean, etc.*
- Control flow: if-elif-else, for, while, continue, break
- Functions
- Others

What is Python?

What is Python?

- Created by Guido van Rossum, and first released in 1990s.
- A general-purpose language: Do almost any kind of programming, e.g., data science, software development, building website, etc.
- Users: Programmers, engineers, researchers, and data scientists across academia and industry.
- Versions: Python 2 (no longer supported since 2020) vs Python 3 (used in this course).
- Free & open source → Large active community

What is Python?

Advantages

- Portable (can run on different machines, e.g., Windows/Linux PCs, Arduino, Raspberry Pi)
- Automatic memory management
- Many packages for doing almost everything (e.g., NumPy, Pandas, Matplotlib)
- Fast code development and test

Disadvantages

- Python code runs a little slower than languages like C/C++
- Multi-threading is hard in Python

Python IDEs

Integrated Development Environment (IDE) makes coding easier

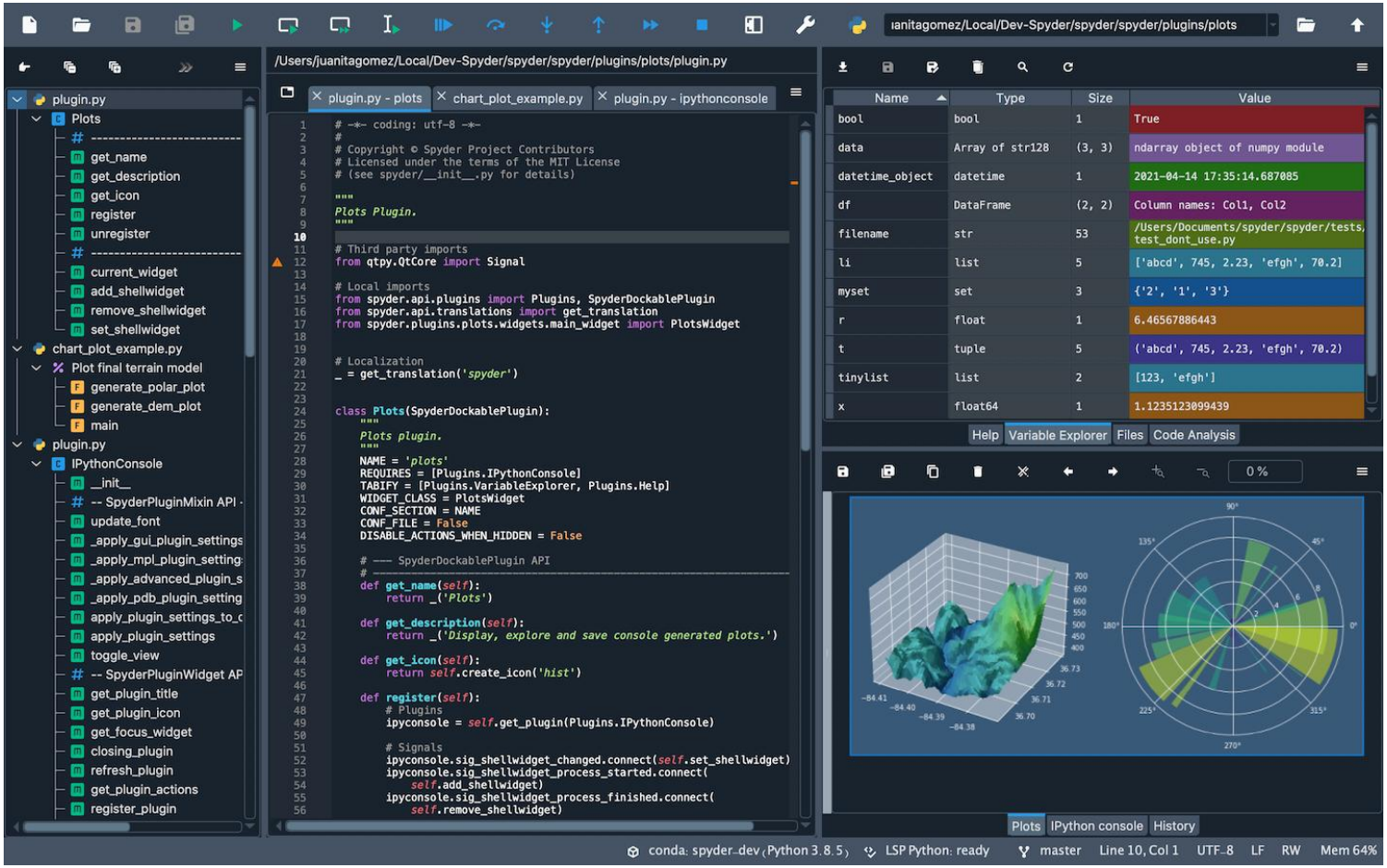
- Save and reload code files
- Run and debug the code
- Syntax highlighting
- Automatic code formatting

Popular free Python IDEs

- Spyder
- VSCODE
- JupyterLab & Colab (web-based)

Python IDEs

Spyder is recommended in this course.



https://docs.spyder-ide.org/current/quickstart.html

Basic syntax

The grammar about how to structure a programming language

Comments are marked by #

```
# set the midpoint
midpoint = 5

# make two empty lists
lower = []; upper = []

# split the numbers into lower and upper
for i in range(10):
    if (i < midpoint):
        lower.append(i)
    else:
        upper.append(i)

print("lower:", lower)
print('upper:', upper)
```

End-of-line terminates a statement

Semicolon can optionally terminate a statement

Code block: Start (**colon**) → Continue (**indentation**)

Parentheses are for grouping or calling

String is enclosed with either single (') or double quotes (")

Python 3: print ("hello") vs Python 2: print "hello"

```
lower: [0, 1, 2, 3, 4]
upper: [5, 6, 7, 8, 9]
```

Data types

Variables

- A variable *assigns* a name to a value.

```
message = "Hello, world!"
n = 42
e = 2.71
print(message)
```

```
"Hello, world!"
```

- Almost always preferred to use variables over values:
  - Easier to update code
  - Easier to understand code (useful naming)

```
print(4.2 * 3.5)
```

```
14.7
```

→

```
length = 4.2
height = 3.5
area = length * height
print(area)
```

```
14.7
```

Data types

- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _) , and it must not start with a number.
- Variable names are case-sensitive (e.g., age, Age and AGE are three different variables).
- Not allowed to use keywords as variable names, which define structure and rules of a language. They include and, or, not, is, in, for, if, elif, else, continue, break, def, class, return, etc. Your IDE will let you know if you try to use one of these.
- Python is dynamically typed, variables are declared by assigning a value that determines their data type.

Data types

Simple data types

Type	Example	Description
int	x = 1	integers (i.e., whole numbers)
float	x = 1.0	floating-point numbers (i.e., real numbers)
bool	x = True	Boolean: True/False values
str	x = 'abc'	String: characters or text (single/double-quoted)
complex	x = 1 + 2j	Complex numbers (having the real and the imaginary parts)
NoneType	x = None	Special object indicating nulls
…		

Data types

- You can always check the type of a variable using type() function.

```
x1 = 1; x2 = 1.0; x3 = True
print(type(x1))
print(type(x2))
print(type(x3))
```

```
<class 'int'>
<class 'float'>
<class 'bool'>
```

```
x4 = 'abc'; x5 = 1 + 2j; x6 = None
print(type(x4))
print(type(x5))
print(type(x6))
```

```
<class 'str'>
<class 'complex'>
<class 'NoneType'>
```

- Working with **strings**

➢ "hello"+"world"	"helloworld"	# concatenation
➢ "hello"*3	"hellohellohello"	# repetition
➢ "hello"[0]	"h"	# indexing
➢ "hello"[-1]	"o"	# (from end)
➢ "hello"[1:4]	"ell"	# slicing
➢ len("hello")	5	# size
➢ "e" in "hello"	True	# search

Data types

Collection data types

Type	Example	Description
list	x = [1, 2, 3]	*Ordered* & *changeable* data collection
tuple	x = (1, 2, 3)	*Ordered* & *unchangeable* collection
dict	x = {'a':1, 'b':2, 'c':3}	*Unordered* & *changeable* mapping (key-value)
set	x = {1, 2, 3}	*Unordered* collection of *unique* values

- Round, square, and curly brackets have distinct meanings when it comes to the type of collection produced.
- In ordered collections (i.e., list and tuple), indexing can starts from the beginning (0, 1, …) or from the end (-1, -2, …).

length = 5

	'p'	'r'	'o'	'b'	'e'
index	0	1	2	3	4
negative index	-5	-4	-3	-2	-1

Data types

Example: Collection data types

```
myList = ['a', 1, 3.2]
myTuple = ('a', 1, 3.2)
myDict = {'name': 'a', 'number': 1, 'mark': 3.2}
mySet = {'a', 1, 3.2}
```

Ordered

```
print(myList[0])
print(myTuple[-1])
```

```
a
3.2
```

Unordered

```
print(myDict[0])
```

```
Error
```

Unordered

```
print(mySet[-1])
```

```
Error
```

Changeable

```
myList[1] = 3
myDict['name'] = 'tom'
print(myList)
print(myDict)
```

```
['a', 3, 3.2]
{'name': 'tom', 'number': 1, 'mark': 3.2}
```

Unchangeable

```
myTuple[1] = 3
```

```
Error
```

Data types

- Woking with list

x = ['a', 'b', 'c']; y = [1, 3, 2]

➢ x[1]; x[-1]	'b'; 'c'	# indexing
➢ x[0:2]	['a', 'b']	# slicing
➢ x + y	['a', 'b', 'c', 1, 3, 2]	# concatenation
➢ 'c' in x	True	# search
➢ len(x)	3	# length
➢ x.append('d')	['a', 'b', 'c', 'd']	# add an item to the end
➢ x.insert(1, 'd')	['a', 'd', 'b', 'c']	# inset an item at the specified index.
➢ x.pop()	'c'	# return the last item & remove it from list
➢ x.remove('b')	['a', 'c']	# remove the first occurrence of 'b'

…

Data types

- Woking with dict

```
x = {'name': 'tom', 'age': 20}
```

➢ x['name']	'tom'	# get value using key
➢ x['mark'] = 78	{'name': 'tom', 'age': 20, 'mark': 78}	# add a key-value pair
➢ x['age'] = 25	{'name': 'tom', 'age': 25}	# modify
➢ del x['age']	{'name': 'tom'}	# delete
➢ x.keys()	['name', 'age']	# get all keys
➢ x.values()	['tom', 20]	# get all values
➢ for k, v in x.items(): print(k + ': ' + str(v))	name: tom age: 20	# loop through all key-value pairs

…

Operators

Arithmetic operators

Operator	Name	Description
a + b	Addition	Sum of a and b
a - b	Subtraction	Difference of a and b
a * b	Multiplication	Product of a and b
a / b	Division	Division of a by b
a % b	Modulus	Integer remainder after division of a by b
a // b	Floor division	Division → Round down to the nearest integer
a ** b	Exponentiation	a raised to the power of b
…		

```
# add, subtract, multiply, exponent
print((4 + 8) * (6.5 - 3) + 3 ** 2)
```

51.0

```
# True division, Floor division, modulus
print(11 / 4, 11 // 4, 11 % 4, end=' ')
```

2.75, 2, 3

Operators

Assignment operators

Operator	Name	Description
=	Assign	x = y + z
+=	Add and assign	a += b → a = a + b
-=	Subtract and assign	a -= b → a = a - b
*=	Multiply and assign	a *= b → a = a * b
/=	Divide and assign	a /= b → a = a / b
**=	Exponent and assign	a **= b → a = a ** b
%=	Modulus and assign	a %= b → a = a % b
…		

- The expression a ■= b is equivalent to a = a ■ b.

```
a = 24
a += 2  # equivalent to a = a + 2
print(a)
```

26

Operators

Comparison operators

Operator	Description	Operator	Description
a == b	a equal to b	a != b	a not equal to b
a < b	a less than b	a > b	a greater than b
a <= b	a less than or equal to b	a >= b	a greater than or equal to b

- Comparison operators return Boolean values True and False.

```
print(20 == 20)

print(20 > 20)
```

```
True
False
```

```
print('apple' != 'apple')

print('apple' != 'orange')
```

```
False
True
```

```
print(20 >= 10)

print(30 < 20)
```

```
True
False
```

Operators

Boolean operators

- When working with Boolean values, Python provides operators to combine the values using operators of and, or, and not.

```
a = 20
print(0 < a and a < 100)
print(0 < a < 100)
```

```
True
True
```

```
x = 4
print((x < 6) and (x > 2))
print((x > 10) or (x % 2 == 0))
print(not (x < 6))
```

```
True
True
False
```

Membership operators

- Membership operators (in and not in) check for membership within compound data (e.g., list and tuple).

```
a = [1, 2, 3]
b = 1
print(b in a)
```

```
True
```

```
a = [1.5, 2.2, 3.9]
b = 1.5
print(b not in a)
```

```
False
```

Control flow

- Control flow is the order in which the program's code executes. It is one of the most important topics related to programming.
- **Conditional statements (`if`-`elif`-`else`)** execute different blocks of codes depending on whether a given condition is true or false.
- **Loop statements (`for` and `while`)** allow the program to execute a block of code repeatedly as long as the condition is true.
- **Transfer statements (`continue` and `break`)** alter the way of execution in a certain manner.

Control flow

Conditional statements: if-elif-else

- In these conditional clauses, elif and else blocks are optional.
- You can optionally include as few or as many elif statements as you would like.
- *Colon* helps the code flow into the following *indented* block.

```
number = 6
if number > 5:
    # Calculate square
    print(number * number)
print('Next lines of code')
```

```
36
Next lines of code
```

```
x = -15

if x == 0:
    print(x, "is zero")
elif x > 0:
    print(x, "is positive")
elif x < 0:
    print(x, "is negative")
else:
    print(x, "is unlike anything I've ever seen...")
```

```
-15 is negative
```

Control flow

Loop statements: for

- We specify the **variable** we want to use, the **sequence** we want to loop over, and use the in operator to link them together.
- "For all elements in a sequence, do something"
- *Colon* helps the code flow into the following *indented* block.

```
for i in [2, 3, 5, 7]:
    print(i*2, end=' ') # print all on same line
```

```
4 6 10 14
```

- The range object is one of the most commonly-used iterators to generate a sequence of numbers.

```
for i in range(10):
    print(i, end=' ')
```

```
0 1 2 3 4 5 6 7 8 9
```

Control flow

Loop statements: while

- The while loop iterates until some condition is met.
- "While this is true, do this."
- *Colon* helps the code flow into the following *indented* block.

```
i = 0
while i < 10:
    print(i, end=' ')
    i += 1
```

```
0 1 2 3 4 5 6 7 8 9
```

Control flow

Transfer statements: continue and break

- They are two useful statements that can be used within (for or while) loops to fine-tune how they are executed.
- The continue statement skips the remainder of the current loop, and goes to the next iteration.
- The break statement breaks out of the loop entirely.

```
names = ["Jimmy", "Rose", "Max", "Nina", "Phillip"]

for name in names:

    if len(name) != 4:
        continue

    print(f"Hello, {name}")

    if name == "Nina":
        break

print("Done!")
```

Control flow

Example: Skip the even numbers (continue)

```
for n in range(20):
    # if the remainder of n / 2 is 0, skip the rest of the loop
    if n % 2 == 0:
        continue
    print(n, end=' ')
```

```
1 3 5 7 9 11 13 15 17 19
```

Example: Break infinite loop (break)

```
i = 1
while True:
    print('6 * ',(i), '=',6 * i)
    if i >= 5:
        break
    i = i + 1
```

```
6 * 1 = 6
6 * 2 = 12
6 * 3 = 18
6 * 4 = 24
6 * 5 = 30
```

Functions

- A function is a block of code that has a name and only runs when it is called using parentheses.
- They are really useful if you have operations that need to be done repeatedly; i.e. calculations.
- The function must be defined before it is called.

Use built-in functions

- Built-in functions are already defined in Python (https://docs.python.org/3/library/functions.html).

```
print('abc')
```

```
abc
```

```
len([1, 3, 5])
```

```
3
```

```
abs(-89)
```

```
89
```

```
max([2, 8, 0])
```

```
8
```

```
round(2.6785, 2)
```

```
2.68
```

Define functions

- Functions become even more useful when we begin to define our own, organizing functionality to be used in multiple places.
- In Python, functions are defined with the def statement.
- *Colon* helps the code flow into the following *indented* block.

Function name

Function parameter(s)

def keyword

```
def celsius_to_kelvin(temp_c):

    temp_k = temp_c + 273.15

    return temp_k

x = celsius_to_kelvin(20.5)
print(x)
```

Colon

Function code(s)

return statement

```
293.65
```

Functions

- We can return multiple values from a function. The values can be received in a tuple variable or an equal number of individual variables.

```
def square_cube(x):
    square = x * x
    cube = x * x * x
    return square, cube

# received in a tuple variable
y = square_cube(5)
print(y)
print(type(y))

# received in individual variables
s, c = square_cube(5)
print("Square = ", s)
print("Cube = ", c)
```

```
(25, 125)
<class 'tuple’>
Square = 25
Cube = 125
```

Default argument values

- We can give default value(s) for parameter(s) in the function definition using the assignment operator (=).
- Such parameters are *optional* during a function call. If no argument is provided, the default value is used; if an argument is provided, it will overwrite the default value.

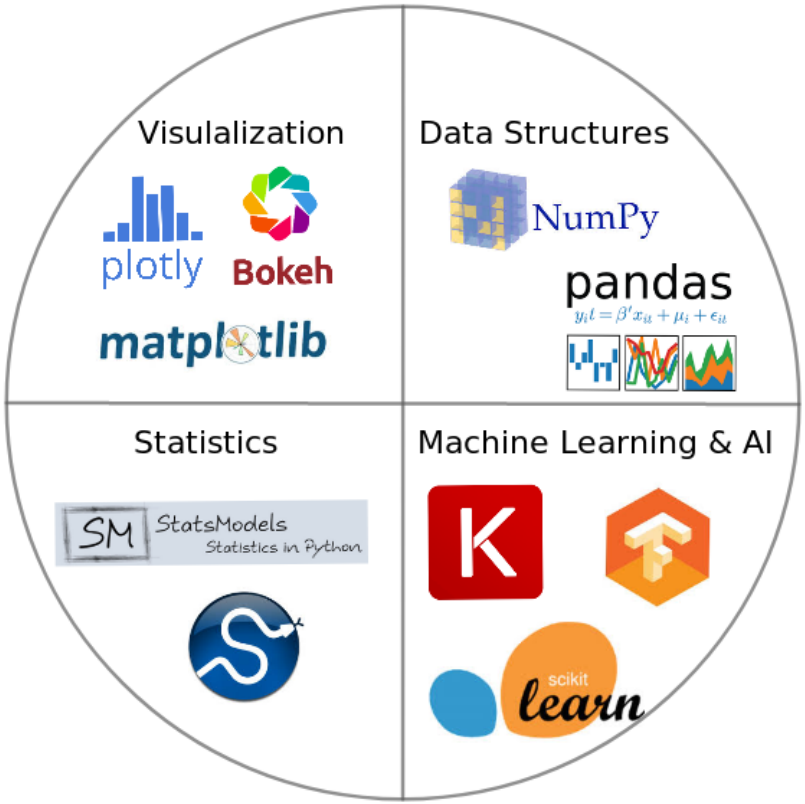
```
def findvolume(length, width=1, depth=1):
    print("[length, width, depth] = ", [length, width, depth])
    volume = length * width * depth
    return volume

v1 = findvolume(10); print("v1 = ", v1)
v2 = findvolume(10, width=15); print("v2 = ", v2)
v3 = findvolume(10, width=12, depth=25); print("v3 = ", v3)
```

```
[length, width, depth] = [10, 1, 1]
v1 = 10
[length, width, depth] = [10, 15, 1]
v2 = 150
[length, width, depth] = [10, 12, 25]
v3 = 3000
```

Others

Python packages

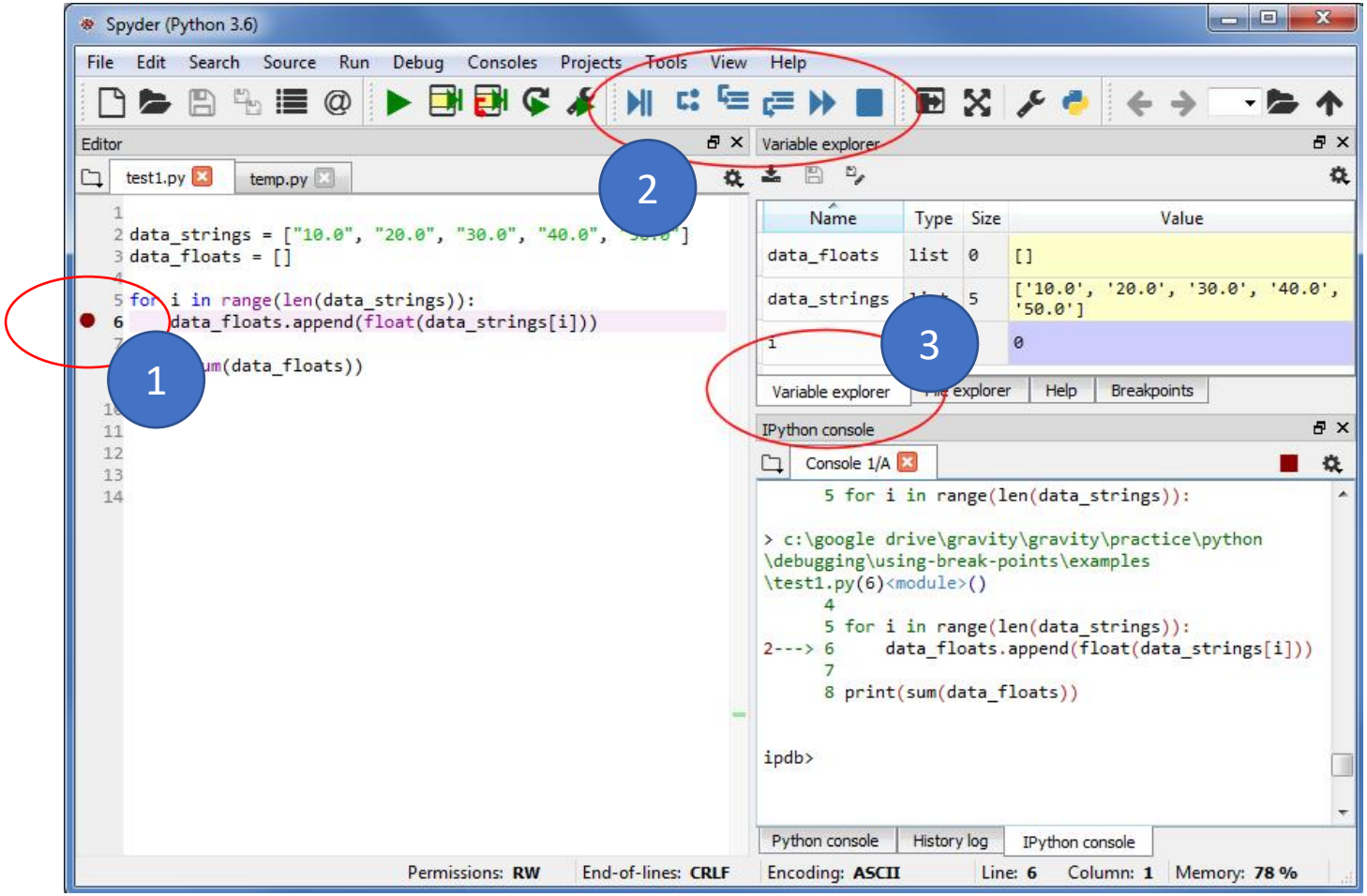
- Package is a collection of files (called modules) that contains functions that can be used repeatedly in different programs.
- *Python Package Index (PyPI)* can help you find and install packages developed and shared by the Python community (https://pypi.org/).
- Python ecosystem for data science/engineering, including *NumPy*, *Pandas*, *Matplotlib, etc.*



Others

Debugging

- Debugging is essential in coding. You will probably spend more time debugging than actually writing code.
- EVERYONE has to debug, it is nothing to be ashamed of.
- When debugging, the most important function at your disposal is `print()`. Every coder uses this as a debugging tool, regardless of their amount of experience.
- For beginners, most IDEs like Spyder include some debugging tools that allow you to step through your script line-by-line to attempt to find an error.

Others

- Steps for debugging in Spyder: 1. Set breakpoint(s) → 2. Run debug tools → 3. Inspect variable explorer